

Java Backend Architecture

Interview Series

Chapter 17.3

Services & Cluster DNS

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.3 - Services and Cluster DNS

Overview

Kubernetes Services provide stable network identities for ephemeral pods. CoreDNS enables service discovery by name. Understanding Service types and DNS resolution is critical for building Java microservices that communicate reliably across the cluster.

Key Definitions

ClusterIP	Default; stable virtual IP reachable only within cluster; DNS: svc.ns.svc.cluster.local.
NodePort	Exposes service on each node IP at port 30000-32767; includes ClusterIP.
LoadBalancer	Provisions cloud LB; external IP assigned (AWS NLB, GCP GCLB); includes NodePort.
ExternalName	DNS CNAME alias to external service DNS; no proxying, no ClusterIP.
Headless Service	clusterIP: None; DNS returns individual pod IPs directly; StatefulSets.
EndpointSlice	Scalable replacement for Endpoints; groups pod IPs in slices of max 100.
CoreDNS	Cluster DNS server; resolves svc-name.ns.svc.cluster.local to ClusterIP.
kube-proxy	Programs iptables/IPVS rules on every node to implement Service VIP routing.
externalTrafficPolicy	Local: preserves client IP, routes to local node pods. Cluster: load balances all pods.
sessionAffinity	ClientIP: routes requests from same client IP to same pod for stickiness.

Diagram 1: Service Types Comparison

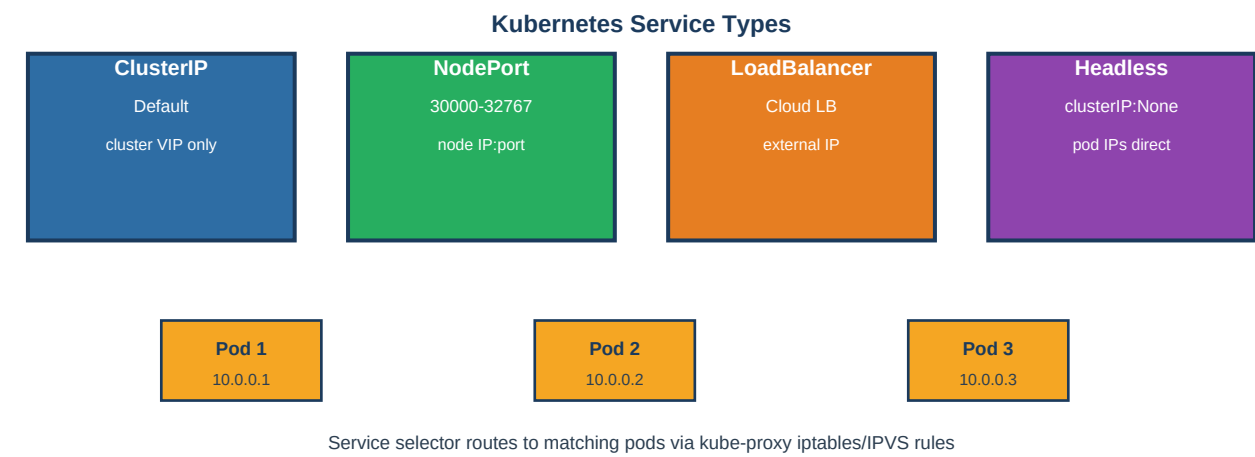
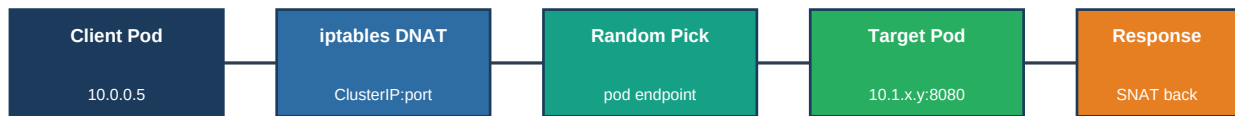


Diagram 2: kube-proxy iptables Flow

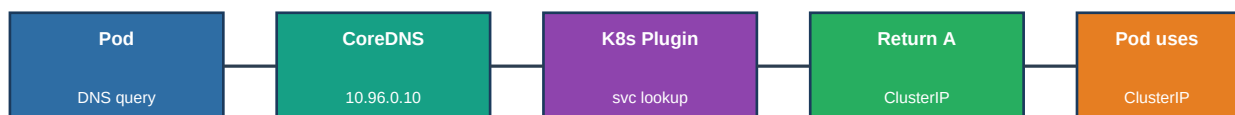
kube-proxy iptables Flow: ClusterIP to Pod



IPVS: kernel hash table O(1) vs iptables O(n) sequential rule traversal

Diagram 3: CoreDNS Resolution Flow

CoreDNS Resolution Flow



`svc-name.namespace.svc.cluster.local` -> ClusterIP

Headless: FQDN -> pod IPs | ExternalName: CNAME to external DNS

Service Types Comparison Table

Type	ClusterIP	External Access	DNS Result	Use Case
ClusterIP	Auto-assigned	No	<code>svc.ns.svc.cluster.local</code>	Internal microservices
NodePort	Yes	NodeIP:30000+	Same as ClusterIP	Dev/test, bare metal
LoadBalancer	Yes	External LB IP	Same as ClusterIP	Production external traffic
ExternalName	No	CNAME redirect	CNAME to external	External service alias
Headless	None	No	Returns pod IPs	StatefulSet, service mesh

Code Examples

ClusterIP Service:

```
apiVersion: v1 kind: Service metadata: name: java-api namespace: default spec: type: ClusterIP
selector: app: java-api ports: - name: http port: 80 targetPort: 8080 - name: grpc port: 9090
targetPort: 9090 # DNS: java-api.default.svc.cluster.local
```

Headless Service for StatefulSet:

```
apiVersion: v1 kind: Service metadata: name: kafka-headless spec: clusterIP: None selector:
app: kafka ports: - port: 9092 targetPort: 9092 # Pod DNS:
kafka-0.kafka-headless.default.svc.cluster.local
```

LoadBalancer with AWS NLB:

```
apiVersion: v1 kind: Service metadata: name: java-api-public annotations:
service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
```

```
service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing" spec: type:
LoadBalancer externalTrafficPolicy: Local selector: app: java-api ports: - port: 443
targetPort: 8443
```

Interview Q&A; - Services and Cluster DNS

Q1. What is a Kubernetes Service and why is it needed?

Pods are ephemeral - they get new IPs on every restart. A Service provides a stable virtual IP (ClusterIP) and DNS name that remains constant even as pods are replaced. The Service selector matches pods by labels; kube-proxy maintains iptables/IPVS rules to distribute traffic from the ClusterIP to matching pod IPs.

Q2. Explain the difference between ClusterIP, NodePort, and LoadBalancer.

ClusterIP: virtual IP only within cluster - for internal microservice communication. NodePort: extends ClusterIP, additionally exposes port 30000-32767 on every node IP. LoadBalancer: extends NodePort, provisions cloud load balancer (AWS NLB/ELB, GCP GCLB) and assigns external IP - for production external traffic.

Q3. What is a headless service and how does DNS work for it?

Headless Service (clusterIP: None) has no virtual IP. CoreDNS returns A records for each pod IP directly. For StatefulSets: pod-0.svc.ns.svc.cluster.local resolves to pod-0 IP. For headless with selector, DNS returns all pod IPs. Used by Kafka/Cassandra for direct pod addressing.

Q4. How does CoreDNS resolve service names?

Pods have /etc/resolv.conf with nameserver CoreDNS IP and search domains: default.svc.cluster.local, svc.cluster.local, cluster.local. Query for "java-api" appends search domains -> java-api.default.svc.cluster.local. CoreDNS K8s plugin returns ClusterIP from Service object. Full FQDN always works.

Q5. What is EndpointSlice vs Endpoints?

Endpoints: one object holds all pod IPs for a Service - large at scale (500 pods = 500 IPs). EndpointSlice (default since 1.21): groups pod IPs in slices of max 100. Benefits: smaller objects, only affected slice updated on pod change (reduces etcd write amplification), dual-stack support (IPv4+IPv6), topology hints for zone-aware routing.

Q6. Explain externalTrafficPolicy: Local vs Cluster.

Cluster (default): routes NodePort/LoadBalancer traffic across all pods in cluster, performs SNAT. Local: routes only to pods on receiving node, no SNAT (preserves client IP). Local may cause uneven load but enables client IP logging. Use Local with LoadBalancer when client IP needed for audit/geo-routing.

Q7. How does session affinity work?

spec.sessionAffinity: ClientIP routes requests from same source IP to same pod. sessionAffinityConfig.clientIP.timeoutSeconds: 10800 (3 hours default). Implemented via iptables recent module tracking source IPs. Not suitable for HTTPS from LB (same source IP). For sticky sessions: use Ingress nginx cookie affinity.

Q8. What DNS records does CoreDNS create?

For ClusterIP Service: A record svc.ns.svc.cluster.local -> ClusterIP, SRV records for named ports. For Headless: A records for each pod IP. SRV format: _port._proto.svc.ns.svc.cluster.local. For pods: pod-ip.ns.pod.cluster.local. Kafka clients use pod FQDN for direct broker connections.

Q9. How do you expose a Service without a cloud LB?

NodePort: clients connect to nodeIP:30000-32767. Need external LB or DNS pointing to nodes. Ingress with Nginx: Nginx pod exposed via NodePort, routes by host/path. MetalLB (bare metal): implements LoadBalancer type using BGP or ARP for service IP announcement. hostNetwork: true - pod uses node network namespace (risk of port conflicts).

Q10. What is ExternalName service?

ExternalName creates a CNAME DNS record pointing to external hostname, no proxying or VIP. Example: db Service -> RDS endpoint `rds.us-east-1.amazonaws.com`. Apps query `db.default.svc.cluster.local`, resolver returns CNAME to RDS. Enables easy migration: change ExternalName to ClusterIP when DB moves into cluster.

Q11. How does kube-proxy handle service updates?

kube-proxy watches Service and EndpointSlice objects via informers. On change: iptables mode: re-runs iptables-restore with updated rules (atomic replacement). IPVS mode: updates virtual server and real server entries via netlink. Eventually consistent - brief period between pod termination and rule removal. Readiness probes critical: pod removed from endpoints before iptables updates.

Q12. Explain Service CIDR and Pod CIDR.

Service CIDR (`--service-cluster-ip-range`, default `10.96.0.0/12`): virtual IP range for ClusterIPs. These IPs never appear on any physical interface; exist only in iptables/IPVS rules. Pod CIDR: actual IP range for pod interfaces, configured by CNI. Each node gets a subnet (e.g., `10.244.1.0/24`). Service CIDR and Pod CIDR must not overlap.

Q13. How does traffic routing work with multiple Pods behind a Service?

iptables: probabilistic DNAT rules. For 3 pods: 1/3 probability each via statistics module. IPVS: configurable algorithms: rr (round-robin), lc (least connections), sh (source hash), dh. Session affinity overrides random selection. Unhealthy pods removed from EndpointSlice - no traffic routed.

Q14. What is topology-aware routing for Services?

TopologyAwareHints (K8s 1.24+): CoreDNS and kube-proxy prefer pod endpoints in same zone. EndpointSlice controller annotates endpoints with zone hints. kube-proxy routes to same-zone pods first. Falls back to other zones if insufficient local capacity. Reduces cross-zone traffic costs on cloud providers (inter-AZ data transfer fees).

Q15. How do you configure internal DNS for custom domain names?

CoreDNS Corefile customized via ConfigMap in kube-system. Stub zones: `example.internal { forward . 10.10.0.53 }` routes to corporate DNS. Rewrite plugin: `rewrite name exact app.internal app.default.svc.cluster.local`. Log plugin: logs DNS queries for debugging. hosts plugin: static IP mappings.

Q16. What is the difference between port, targetPort, nodePort?

port: port on Service (ClusterIP:port) that clients connect to. targetPort: port on pod container to forward traffic to (can be named port). nodePort: static port 30000-32767 on every node (NodePort/LoadBalancer only). Example: port:80, targetPort:8080, nodePort:31000 -> client:80, pod:8080, external:31000.

Q17. How does a Service know which pods to route to?

Service spec.selector matches against pod metadata.labels. EndpointSlice controller watches pods, creates EndpointSlice entries for matching pods that are Running and have passed readinessProbe. If pod fails readiness, removed from EndpointSlice -> no traffic. Headless with no selector: manually create Endpoints with custom IP list.

Q18. Explain DNS TTL behavior in Kubernetes.

CoreDNS returns Service A records with TTL 5 seconds by default. Java apps should respect TTL and re-resolve; not cache DNS indefinitely. JVM default DNS cache: 30s (positive), 10s (negative). Set `networkaddress.cache.ttl=5` for K8s deployments to pick up pod IP changes quickly.

Q19. What is a multi-port Service?

Service can expose multiple ports: specify list under `spec.ports` with names. Example: `java-api` exposes port 8080 (HTTP) and port 9090 (gRPC) on same ClusterIP. SRV DNS records created per named port: `_http._tcp.java-api.ns.svc.cluster.local`. Useful for apps serving both REST and gRPC (Spring Boot with gRPC server).

Q20. How does LoadBalancer service interact with cloud providers?

cloud-controller-manager Service controller watches LoadBalancer Services. On creation: calls cloud API to provision LB, configures health checks via NodePort, sets `service.status.loadBalancer.ingress.ip/hostname`. Annotations customize LB: AWS internal/external, TLS, connection draining. On deletion: cloud LB is deprovisioned automatically.

Q21. What is the DNS search path and how does it affect latency?

Default search: `[namespace].svc.cluster.local`, `svc.cluster.local`, `cluster.local`, `node-domain`. For short name "db": tries `db.[namespace].svc.cluster.local` first. For non-existent names: each search domain gets a DNS query (NXDOMAIN). Use full FQDN (`db.default.svc.cluster.local`) for deterministic resolution. `ndots:5` setting means FQDNs with 5+ dots tried first without appending search domains.

Q22. How does kube-proxy handle pod termination?

Pod starts terminating: (1) EndpointSlice controller removes pod from slice. (2) kube-proxy updates iptables/IPVS to stop routing to that pod. (3) kubelet sends SIGTERM to container. Race condition: new connections may briefly hit terminating pod before iptables update. Mitigation: `preStop` hook with sleep delays SIGTERM until iptables propagates.

Q23. What is publishNotReadyAddresses in headless Services?

By default headless Service DNS only returns ready pod IPs. `spec.publishNotReadyAddresses: true` includes all pods (even not-ready) in DNS results. Critical for StatefulSet clustering: pod-0 needs pod-1 DNS before pod-1 fully ready (Cassandra seed discovery, ZooKeeper peer discovery). Without this, pods may fail to join cluster during initial bootstrap.

Q24. How do you troubleshoot a Service not routing traffic?

`kubectl get endpoints svc-name` (should show pod IPs; if empty: selector mismatch or pod not Ready). `kubectl describe service svc-name`: check selector and port configuration. `kubectl exec pod -- curl svc-name.namespace:port`. `kubectl exec pod -- nslookup svc-name.namespace.svc.cluster.local`. `kubectl logs -n kube-system kube-proxy-xxx` for proxy errors.

Q25. What is an ExternalIP Service?

`spec.externalIPs: [1.2.3.4]` makes kube-proxy listen on that external IP on all nodes. Traffic to `1.2.3.4:port` is DNAT-ed to service pods. Must own and route that IP to your nodes (BGP or static routes). Different from LoadBalancer (which provisions the external IP from cloud). Use: bare metal clusters with static public IPs, BGP-routed IP blocks.

Q26. How does Spring Cloud Kubernetes integrate with K8s Services?

`spring-cloud-kubernetes-client-discovery` uses K8s API to list Services/Endpoints as service instances. Spring Feign with `@LoadBalanced` uses K8s DNS or service registry. ConfigMap support: `spring.cloud.kubernetes.config.name=app-config` loads K8s ConfigMap. For most cases: use K8s DNS directly (`svc-name.namespace`) without Spring Cloud K8s overhead.

Q27. Explain how IPVS improves upon iptables mode.

iptables: rules evaluated sequentially per packet. 10000 Services * 3 pods = 30000+ rules, O(n). IPVS: kernel hash table, O(1) lookup regardless of Service count. Algorithms: rr, lc (least connections), dh, sh, sed, nq. Better connection tracking, lower CPU at high packet rates. Enable: `kube-proxy --proxy-mode=ipvs` (requires

ipvs kernel modules).

Q28. What is CoreDNS negative caching?

CoreDNS caches NXDOMAIN responses with TTL from SOA record (default 30s). If Service is created after pod has cached NXDOMAIN, pod sees DNS failure for 30s. Workaround: reduce negative TTL in Corefile: `cache { denial 5 }`. Or pre-create Service before pods start. Java: set `networkaddress.cache.negative.ttl=5` JVM property.

Q29. What is the DNS format for StatefulSet pods?

Format: `pod-name.headless-service.namespace.svc.cluster.local` Example: `kafka-0.kafka-headless.default.svc.cluster.local` -> kafka-0 pod IP. This resolves to the specific pod (not load balanced). Kafka brokers advertise this FQDN as listener address. Clients always reach the specific pod regardless of rescheduling.

Q30. How does Kubernetes implement DNS-based service discovery for Java apps?

Spring Boot datasource: `spring.datasource.url=jdbc:postgresql://postgres.default:5432/mydb`. Kafka bootstrap: `spring.kafka.bootstrap.servers=kafka-0.kafka-headless:9092`. Redis: `spring.redis.host=redis.default`. JVM DNS resolution uses `/etc/resolv.conf` search path. Ensure Java respects short TTL: `networkaddress.cache.ttl=5`.

Q31. What happens to existing connections when pod removed from endpoints?

When pod removed from EndpointSlice: existing TCP connections are NOT terminated immediately. kube-proxy removes DNAT rule - no NEW connections routed to that pod. Existing connections continue until closed by app or TCP timeout. readinessProbe should fail before SIGTERM to give connections time to drain. `terminationGracePeriodSeconds` must be long enough for in-flight requests to complete.

Q32. What is InternalTrafficPolicy?

`spec.internalTrafficPolicy`: Local routes cluster-internal traffic only to pods on same node. Cluster (default): routes to any pod in cluster. Use: DaemonSet-backed services where each pod should only see data from its own node. If no local pod with Local policy: connection dropped (no fallback to other nodes). Different from `externalTrafficPolicy` (applies to external/NodePort traffic only).

Q33. How does dual-stack Kubernetes networking work?

K8s supports IPv4 and IPv6 simultaneously. Service gets both IPv4 and IPv6 ClusterIPs. `ipFamilyPolicy`: SingleStack, PreferDualStack, RequireDualStack. Requires dual-stack CNI and `--service-cluster-ip-range` for both families. Java: bind server to `::` (all interfaces) to accept both IPv4 and IPv6 connections.

Q34. What is NodeLocal DNSCache?

NodeLocal DNSCache runs a CoreDNS instance as DaemonSet on each node. Pods are configured to query the local cache (169.254.20.10) instead of CoreDNS ClusterIP. Benefits: eliminates cross-node DNS queries, removes UDP conntrack table exhaustion bug at scale, reduces DNS query latency. The local cache forwards cache misses to CoreDNS upstream.

Q35. How do you implement rate limiting at the Service level?

K8s Services do not natively support rate limiting. Options: Ingress + Nginx: `nginx.ingress.kubernetes.io/limit-rps` annotation. Istio VirtualService: rate limiting via EnvoyFilter or external auth. API Gateway (Kong, Ambassador): rate limit per consumer/service. Application-level: Resilience4j RateLimiter in Java service.

Q36. What are security implications of NodePort services?

NodePort opens port on every node, even nodes without matching pods. Exposed to anyone who can reach node IPs. Best practices: restrict NodePort range (--service-node-port-range), use NetworkPolicies to restrict access, use cloud security groups to restrict node port access. Prefer LoadBalancer or Ingress with WAF for external-facing services.

Q37. How do you configure health checks for an AWS LoadBalancer Service?

AWS NLB health checks via annotations: service.beta.kubernetes.io/aws-load-balancer-healthcheck-protocol: HTTP service.beta.kubernetes.io/aws-load-balancer-healthcheck-path: /actuator/health service.beta.kubernetes.io/aws-load-balancer-healthcheck-interval: 30 AWS Load Balancer Controller (Ingress -> ALB) provides more fine-grained control.

Q38. What is the appProtocol field in Service ports?

appProtocol hints at application protocol: http, https, grpc, mongo, mysql, redis. Used by: Ingress controllers for HTTP/2 upgrade decisions, AWS LB Controller for target group protocol, Istio for L7 traffic management (gRPC-specific routing, retries, metrics). Best practice: always set appProtocol for named ports.

Q39. How does CoreDNS autoscaling work?

CoreDNS Deployment with CPA (Cluster Proportional Autoscaler). CPA scales based on cluster size: nodesPerReplica: 16, coresPerReplica: 256. For 1000+ nodes: 20+ CoreDNS replicas needed. NodeLocal DNSCache: CoreDNS as DaemonSet on each node - eliminates cross-node DNS queries, prevents UDP NAT connection tracking bugs at scale.

Q40. What is a Service without selector?

Service without spec.selector: K8s does not auto-create Endpoints. Create Endpoints/EndpointSlice manually pointing to external IPs. Use: migrate external service to K8s gradually - Service name stays same, endpoints switch from external IP to pod IPs when migration complete. Also: multi-cluster routing to other cluster pod IPs.

Q41. How do you configure ClusterIP services for cross-namespace access?

Short name "svc" resolves in same namespace. Cross-namespace: use svc.other-namespace or FQDN svc.other-namespace.svc.cluster.local. NetworkPolicy may block traffic even if DNS resolves - add ingress rule in target namespace. RBAC does not restrict DNS - any pod can resolve any service name in any namespace.

Q42. What is the trafficDistribution field?

spec.trafficDistribution: PreferClose (K8s 1.31+) is a simpler alternative to topology hints. Prefers routing to endpoints in same zone/region as the client. Does not require manual hint configuration like TopologyAwareHints. Falls back to routing anywhere if no local endpoints available. Simpler to configure than per-zone EndpointSlice hints.

Q43. How does the AWS Load Balancer Controller differ from classic ELB provisioning?

Classic (cloud-controller-manager): creates CLB/NLB per LoadBalancer Service - one LB per service. AWS Load Balancer Controller: Ingress -> ALB (path/host routing, one ALB for many services), Service -> NLB (target group binding). IP mode (pod IPs as targets) vs instance mode (node IPs). IP mode: lower latency, no NodePort hop.

Q44. What is a service mesh vs K8s Services?

Service mesh (Istio, Linkerd) adds L7 capabilities: mTLS between services (automatic cert rotation), traffic management (canary routing, fault injection, circuit breaking via VirtualService/DestinationRule), observability (distributed tracing, per-service metrics). Implemented via Envoy sidecar in each pod. K8s Services provide initial discovery; mesh handles L7 policies.

Q45. How does Kubernetes Services handle pod readiness updates?

readinessProbe runs at periodSeconds interval. On failure: kubelet sets pod Ready=False. EndpointSlice controller removes pod from slice. kube-proxy removes iptables/IPVS rule for pod. New traffic no longer routed. Pod remains Running (not restarted). When readiness passes: pod re-added to EndpointSlice, starts receiving traffic.

Q46. What are named ports and why use them?

Named ports allow referencing ports by name in Ingress backend (servicePort: http), PrometheusServiceMonitor (port: metrics), HPA custom metrics. Decoupling: if container port number changes, only update container - Service still references "http". SRV DNS: _http._tcp.svc.ns.svc.cluster.local. Best practice: always name ports in Service and container specs.

Q47. How do you debug DNS issues in Kubernetes?

kubectl exec -it pod -- nslookup svc-name.ns.svc.cluster.local 10.96.0.10 (test against CoreDNS IP directly).
kubectl exec pod -- cat /etc/resolv.conf (check search domains and nameserver). kubectl logs -n kube-system -l k8s-app=kube-dns (CoreDNS logs). Enable CoreDNS log plugin: add "log" to Corefile. NetworkPolicy blocking UDP 53 to kube-system is a common cause of DNS failures.

Q48. What is CPA (Cluster Proportional Autoscaler) for CoreDNS?

CPA automatically scales CoreDNS replicas proportional to cluster size. ConfigMap settings: nodesPerReplica (scale 1 replica per N nodes), coresPerReplica (scale 1 replica per N CPU cores), min and max replica bounds. Different from HPA: uses cluster size metrics instead of resource metrics. Better than HPA for infrastructure components like CoreDNS.

Q49. How do you implement circuit breaking for Kubernetes Services?

K8s Services do not natively support circuit breaking. Options: Istio DestinationRule: outlierDetection (consecutive5xxErrors, interval, baseEjectionTime). Resilience4j CircuitBreaker in Java app: registers with MeterRegistry for Prometheus metrics. Spring Cloud Circuit Breaker: abstracts over Resilience4j/Hystrix. Envoy outlier detection: automatic ejection of unhealthy pods from LB pool.

Q50. What is the significance of the EndpointSlice topology annotation?

EndpointSlice endpoints have hints.forZones annotation set by EndpointSlice controller. When kube-proxy uses topology-aware routing, it prefers endpoints with matching zone hint. Controller adds hints only when traffic is roughly balanced (within 2/3 of average per zone). Annotation key: endpointslice.kubernetes.io/managed-by tracks the controller source. Inspect: kubectl get endpointslices -o yaml to see zone hints.

Q51. How do Services interact with Kubernetes Network Policies?

NetworkPolicy controls pod-to-pod traffic at IP/port level, not at Service level. When pod A connects to Service of pod B: traffic is routed to pod B IP (after DNAT). NetworkPolicy ingress rule on pod B must allow source pod A (by podSelector or namespaceSelector). Service selector and NetworkPolicy are independent. Best practice: default-deny all ingress, explicitly allow per-service communication.

Q52. What is the difference between Endpoints and EndpointSlices in terms of update behavior?

Endpoints: single object updated atomically when any pod changes IP. For 1000 pods: every pod restart rewrites 1000-IP Endpoints object in etcd. EndpointSlice: 10 slices of 100 pods each. Only the slice containing changed pod is updated. 10x reduction in etcd write amplification. kube-proxy watches EndpointSlices by default since K8s 1.21.

Q53. How does Kubernetes handle IPv6-only or dual-stack Services?

spec.ipFamilies: [IPv6] creates IPv6-only Service with IPv6 ClusterIP. spec.ipFamilyPolicy: RequireDualStack assigns both IPv4 and IPv6 ClusterIPs. DNS: A record for IPv4, AAAA record for IPv6. Java apps connecting to IPv6 Services: ensure JVM IPv6 preference: -Djava.net.preferIPv6Addresses=true or use hostname (Java resolves to appropriate record).